

Teaspoon: Liquid Types and More for TypeScript

Nicola Gannon  

University of California, Santa Cruz, USA

Kyle Miller   

University of California, Santa Cruz, USA

Abstract

Static type systems for programming languages provide stronger guarantees of program correctness through analysis at compile time. They typically eliminate all runtime type errors and enforce proper usage of APIs. The programmer can even lean on the type system to help her reason about the nature of her data. The benefits of static typing are best showcased in web development. JavaScript is the executable language of the web, but most[4] web developers use TypeScript, a statically typed extension to JavaScript. TypeScript is a compiler-as-specification, which works remarkably well in practice, but a robust theory would prevent soundness bugs. Teaspoon aims to do what TypeScript did for JavaScript. It extends a subset of TypeScript, formally backing it with the usual operational semantics, type inference, and constraint solving. Teaspoon's type system is unusual in its support for mutability and imperative programming. The addition of first-class propositions and liquid types effectively create a dependently typed TypeScript.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Logic $\rightarrow$ Type Theory; Theory of computation $\rightarrow$ Semantics and reasoning $\rightarrow$ Program semantics $\rightarrow$ Operational semantics

Keywords and phrases Dependent Types, Interactive Theorem Proving, Liquid Types

Category Workshop Report

Funding This material was supported by the juice purchased from the Baskin Engineering Perk on 2026-06-11

1 Introduction

Modern software development has benefitted enormously from static type systems and compile-time analysis. Nowhere has demonstrated this better than web-development. The community at large has moved away from dynamically typed JavaScript and towards TypeScript, a statically typed superset of JavaScript. TypeScript has surprisingly advanced types for a well-adopted industry language. A programmer can create ersatz dependent types using TypeScript's value types and perform calculations on them in an untyped lambda-calculus fashion. Higher Kinded Types can be bootlegged[1] using the powerful `infer` construct.

How can we bring TypeScript even closer to advanced type systems like Lean4?

■ Type Theory Foundations? (Model it on CoC?)/

2 Overview

First, programs are elaborated from an input syntax into the core syntax, then the type inference kicks in. It models things using the Calculus of Constructions [3]

Program 1 Monomorphic Identity

```

1 declare const takesFive:
2   5 → 5;
3 // Γ1.02
4 const id5 = a ⇒ {
5   // Γ1.04
6   takesFive(a);
7   // Γ1.06, C1.06
8   return a;
9   // Γ1.08
10 };
11 // Γ1.10

```

Program 2 Basic Flow Typing

```

1 declare const cond: boolean;
2 let greeting;
3 if (cond) {
4   // Γ2.04
5   greeting = "hello!";
6   // Γ2.06, C2.06
7 } else {
8   // Γ2.04
9   greeting = ["goodbye!"];
10  // Γ2.10, C2.10
11 }
12 // Γ2.12

```

Figure 1 Type Elaboration Examples

3 Elaboration

The primary function of elaboration is to translate the input syntax (Table appx.1.1) to the core syntax seen in Table 1. It does this by creating metavariables where types are left unspecified. Unlike Lean 4, Teaspoon’s elaboration is not type directed; the elaboration rules can be presented separately from the type inference rules. In the implementation, the elaborator is also responsible for translating the syntax into a locally nameless[2] representation and ensuring that variables are defined before use.

Teaspoon’s behavior when inferring the type of variables is different than TypeScript’s. Teaspoon allows Program 3a, and rejects Program 3b. In TypeScript, both programs are equivalent and are rejected. TypeScript immediately assigns the absolute type of a variable at the first opportunity. E-LetDef, on the other hand, does not claim to know an absolute type for a variable without a type ascription. If the user wants to restrict the domain of her variable, she may explicitly annotate it (E-LetDecT) as seen in program b. E-Const matches the behavior of TypeScript.

Program 3 Absolute Type of Variable

(a)	(b)
let x = 1;	let x: number = 1;
x = "hello"	x = "hello";

$$\begin{array}{c}
\frac{\tau = \text{unknown}}{\llbracket \text{let } x; \mathcal{P} \rrbracket \triangleright \text{let } x : \tau; \llbracket \mathcal{P} \rrbracket} \text{E-LetDec} \qquad \frac{\llbracket e \rrbracket \triangleright e'}{\llbracket \text{let } x : e; \mathcal{P} \rrbracket \triangleright \text{let } x : e'; \llbracket \mathcal{P} \rrbracket} \text{E-LetDecT} \\
\\
\frac{\llbracket e \rrbracket \triangleright e' \quad \tau = \text{unknown}}{\llbracket \text{let } x = e; \mathcal{P} \rrbracket \triangleright \text{let } x : \tau; x = e'; \llbracket \mathcal{P} \rrbracket} \text{E-LetDef} \qquad \frac{\llbracket e_1 \rrbracket \triangleright e'_1 \quad \llbracket e_2 \rrbracket \triangleright e'_2}{\llbracket \text{let } x : e_1 = e_2; \mathcal{P} \rrbracket \triangleright \text{let } x : \tau; x = e'; \llbracket \mathcal{P} \rrbracket} \text{E-LetDefT} \\
\\
\frac{\llbracket e \rrbracket \triangleright e' \quad \text{fresh } ?m}{\llbracket \text{const } x = e; \mathcal{P} \rrbracket \triangleright \text{const } x : ?m = e'; \llbracket \mathcal{P} \rrbracket} \text{E-Const} \qquad \frac{\text{fresh } ?m}{\llbracket P x \rrbracket \triangleright P x : ?m} \text{E-Param} \\
\\
\frac{\llbracket e \rrbracket \triangleright e'}{\llbracket P x : e \rrbracket \triangleright P x : e'} \text{E-ParamT} \qquad \frac{\llbracket e \rrbracket \triangleright e' \quad \text{fresh } ?m}{\llbracket x \Rightarrow e \rrbracket \triangleright (x : ?m) \Rightarrow e'} \text{E-Lam1} \\
\\
\frac{\llbracket P p \rrbracket \triangleright P x : e'_1 \quad \llbracket e_2 \rrbracket \triangleright e'_2}{\llbracket \langle (p, *) \Rightarrow e_2 \rangle \triangleright (x : e'_1) \Rightarrow e'_2 \rrbracket} \text{E-Lam}
\end{array}$$

■ **Figure 2** Elaboration Rules

E-Lam1 handles unary lambdas without parentheses, synthesizing a fresh metavariable for the type of the parameter. E-Lam is superficially complicated: it collects all parameters of the lambda into $\bar{p}$, and elaborates a fake marker syntax $P p$. The parameter p may be annotated with a type $p = x : e$ or not $p = x$. The former will be elaborated by E-Param and the latter E-ParamT.

■ **Table 1** Core Syntax

$e \rightarrow$	expr	$\tau \rightarrow$	typeish
$bool$		Sort u	
$number$		unknown	
$string$		True	true type
undefined		False	false type
x	var	typeof e	
$?m$	mvar	Liquid	
sorry		$((x : e))^*, e'$	liquid type
$((infer))^?$	lambda	$s \rightarrow$	statement
$((x : e,))^* \Rightarrow e'$	call	let $x : e$	var
$e ((e'))^*$	unimplicit	const $x : e = e$	constant
@ e	sub-expr	if (e) b_1	if
(e)	tactics	else b_2	
with $tactics$	bar	loop b	
$e_1 \mid e_2$	and	break	
$e_1 \& e_2$	logical not	return e	
! e		$x = e$	assign
φ		#check e	
τ		#eval e	
$\varphi \rightarrow$	propish	$b \rightarrow \{((s;))^*\}$	block
$e_1 \equiv e_2$	object eq		
$e_1 @ = e_2$	prop eq		
$e_1 \text{ extends } e_2$	subtype		

4 Operational Semantics

Operational semantics are modeled in a big-step way using something similar to capsule environments[5].

$$\frac{\langle \sigma, e \rangle \Downarrow \langle \sigma', e' \rangle}{\langle \sigma, x = e; ((\bar{s});)^* \rangle \Downarrow \langle \sigma' [x \setminus e'], \bar{s} \rangle} \text{O-Assign}$$

$$\frac{\langle \sigma, e \rangle \Downarrow \langle \sigma', e' \rangle}{\langle \sigma, x = e; ((\bar{s});)^* \rangle \Downarrow \langle \sigma' [x \setminus e'], \bar{s} \rangle} \text{O-Call}$$

■ **Figure 3** Big Step Operational Semantics

5 Type Inference

In Teaspoon’s type system, every expression may include function calls which mutate state. Therefore, complete typechecking involves not only learning the type of an expression, but also the variables it clobbers. Consider Program 4: after `z = 2` on line 3, the programmer

■ **Program 4** Context Clobbering

```
1 let z = 1;
2 const hz: z @= 1 = with rfl;
3 z = 2;
```

shouldn’t have access to `z @= 1`—that would be a contradiction. `hz` **must** be clobbered in some way. The type inference system therefore keeps track of all uses of `z` and upon reassignment, will remove them from the context.

$$\frac{\Gamma \vdash}{\langle \sigma, x = e; ((\bar{s}); *) \rangle \Downarrow \langle \sigma' [x \setminus e'], \bar{s} \rangle} \text{T-Assign}$$

■ **Figure 4** Type Inference Rules

With mutable variables something something something.

Have to forget proofs about what they are, and widen to what they can be.

With constants, all new information you learn stays with you.

There’s something similar about how you can’t “keep” information between function calls.

6 Future Work

Though the theory feels sound at the moment and correct so-far, the actual implementation is extremely incomplete. Essentially nothing is *completely* written out, not the elaboration rules, operational semantics, or type inference system. All of it lives in the author’s mind. In that regard, there is immediate, well-specified future work.

There are still open questions, too:

- What should the tactic language look like? How should it be parsed?
- Does `unknown` need a universe level? Will there be paradoxes if it doesn’t have one?
- How should a hoare-like program-logic be embedded?

References

- Giulio Canti. 2019. @gcanti/fp-ts/src/HKT.ts. Retrieved from <https://github.com/gcanti/fp-ts/blob/master/src/HKT.ts>
- 1 Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (October 2012), 363–408. <https://doi.org/10.1007/s10817-011-9225-2>
- 2 Thierry Coquand and Gérard Huet. 1988. The calculus of constructions. *Information and Computation* 76, 2 (February 1988), 95–120. [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3)
- 3 Sacha Greif. State of JS: 2025. Retrieved from https://2025.stateofjs.com/en-US/usage/#js_ts_balance
- 4 Jean-Baptiste Jeannin and Dexter Kozen. 2012. Computing with Capsules. 7386, (2012), 1–19. https://doi.org/10.1007/978-3-642-31623-4_1
- 5

7 Appendix

Table appx.1.1 Input Syntax

$e \rightarrow$	$\begin{array}{l} \text{bool} \\ \text{number} \\ \text{string} \\ \text{undefined} \\ x \\ ?m \\ \text{sorry} \\ - \\ ((\text{infer}))? \\ ((x : e,)) * \Rightarrow e' \\ e ((e')) * \\ @ e \\ (e) \\ \text{with } \textit{tactics} \\ e_1 \mid e_2 \\ e_1 \& e_2 \\ !e \\ \varphi \\ \tau \end{array}$
$\varphi \rightarrow$	$\begin{array}{l} e_1 \equiv e_2 \\ e_1 @ = e_2 \\ e_1 \text{ extends } e_2 \\ e_1 \text{ satisfies } e_1 \end{array}$
$\tau \rightarrow$	$\begin{array}{l} \text{Prop} \\ \text{Type} \\ \text{Sort } u \\ \text{Type } u \\ \text{True} \\ \text{False} \\ \text{typeof } e \\ \text{Liquid } ((x : e)) *, e' \end{array}$
$s \rightarrow$	$\begin{array}{l} \text{let } x : e \\ \text{const } x : e = e \\ \text{if } (e) b \\ \text{if } (e) b_1 \\ \text{else } b_2 \\ \text{loop } b \\ \text{break} \\ \text{return} \\ \text{return } e \\ x = e \\ \# \text{check } e \\ \# \text{eval } e \end{array}$
$b \rightarrow$	$\{((s;)) * \}$